

LeetCode 1876: Substrings of Size Three with Distinct Characters

Detailed Solution and Code Analysis

 **Target Companies: Asked in Visa, Accenture**

Problem Overview

The problem asks us to find the number of **good substrings** of length exactly 3 in a given string s . A string is considered *good* if there are no repeating characters in it.

Approach & Intuition

This problem is an excellent candidate for the **Fixed-Size Sliding Window** technique. Since the window size is strictly 3, we can efficiently maintain the count of characters as a 3-character window slides through the string from left to right.

- **Step 1: Initialization & Base Cases** - We first handle the edge case where the string length is less than 3. If so, it's impossible to form a substring of size 3, so we return 0 immediately. We also set up a `freq` array of size 26 to track occurrences of each alphabetical character.
- **Step 2: The First Window** - We iterate through the first 3 characters, updating our frequency map. We then verify if all characters in this initial window are unique (no frequency exceeds 1). If they are, we increment our answer counter.
- **Step 3: Sliding the Window** - For every subsequent character, we slide the window to the right by one position. We do this in $O(1)$ time by decrementing the frequency of the character that is *leaving* the window (at index $j - 3$) and incrementing the frequency of the new character *entering* the window (at index j).
- **Step 4: Uniqueness Validation** - After each slide, we loop through the 26-element array to check if any character's frequency is strictly greater than 1. If the characters are unique, we add 1 to our answer.

Code Implementation (C++)

```

class Solution {
public:
    int countGoodSubstrings(string s) {
        int ans = 0;

        int n = s.size();
        if(n < 3) return 0;
        vector<int> freq(26, 0);

        // Process the first window of size 3
        for (int i = 0; i < 3; i++) {
            freq[s[i] - 'a']++;
        }

        bool unique = true;
        for (int i = 0; i < 26; i++) {
            if (freq[i] > 1) {
                unique = false;
                break;
            }
        }

        if (unique)
            ans++;

        // Slide the window across the rest of the string
        for (int j = 3; j < n; j++) {
            freq[s[j - 3] - 'a']--;
            freq[s[j] - 'a']++;

            bool unique = true;
            for (int i = 0; i < 26; i++) {
                if (freq[i] > 1) {
                    unique = false;
                    break;
                }
            }

            if (unique)
                ans++;
        }

        return ans;
    }
};

```

```
}  
};
```

Complexity Analysis

Time Complexity: $O(N)$, where N is the length of the string. We iterate through the string in a single pass. The inner loop checking the 26 characters runs in $O(26)$ which simplifies to $O(1)$ constant time.

Space Complexity: $O(1)$. The frequency array strictly takes 26 integer spaces, which is entirely independent of the input string size.